

中山大學
SUN YAT-SEN UNIVERSITY



国家超级计算广州中心
NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

CUDA全局内存

任课教师：吴迪

Roadmap

- Introducing the CUDA Memory Model
- Memory Management
- Memory Access Patterns
- What Bandwidth Can a Kernel Achieve?
- Matrix Addition with Unified Memory

参考阅读：Cheng-Ch4

1. Introducing the CUDA Memory Model

1.1 Benefits of a Memory Hierarchy

Benefits of a Memory Hierarchy

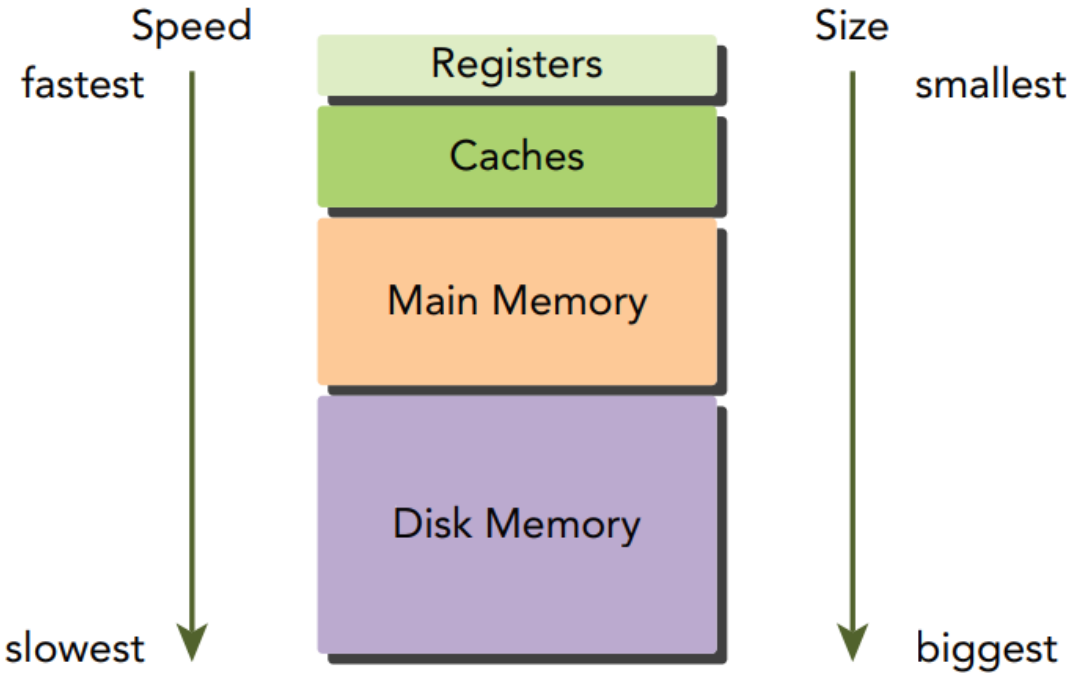
- Two different types of locality:
  - Temporal locality (locality in time)
  - Spatial locality (locality in space)

- **Temporal locality:** if a data location is referenced, then it is more likely to be referenced again within a short time period

- **Spatial locality:** if a memory location is referenced, nearby locations are likely to be referenced as well.

Memory Hierarchy

- This memory hierarchy is only useful because of the principle of locality.
  - A memory hierarchy consists of multiple levels of memory with different latencies, bandwidths, and capacities.



- Lower cost per bit
- Higher capacity
- Higher latency
- Less frequently accessed by the processor

Memory Hierarchy

- This memory hierarchy can provide the illusion of large but low-latency memory

- Both GPUs and CPUs use similar principles and models in memory hierarchy design.

- The key difference is that CUDA programming model exposes more of the memory hierarchy and gives you more explicit control over its behavior.

1.2 CUDA Memory Model

Memory Model

- Two classifications of memory for programmers:
  - **Programmable:** You explicitly control what data is placed in programmable memory.
  - **Non-programmable:** You have no control over data placement .

CPU memory hierarchy:

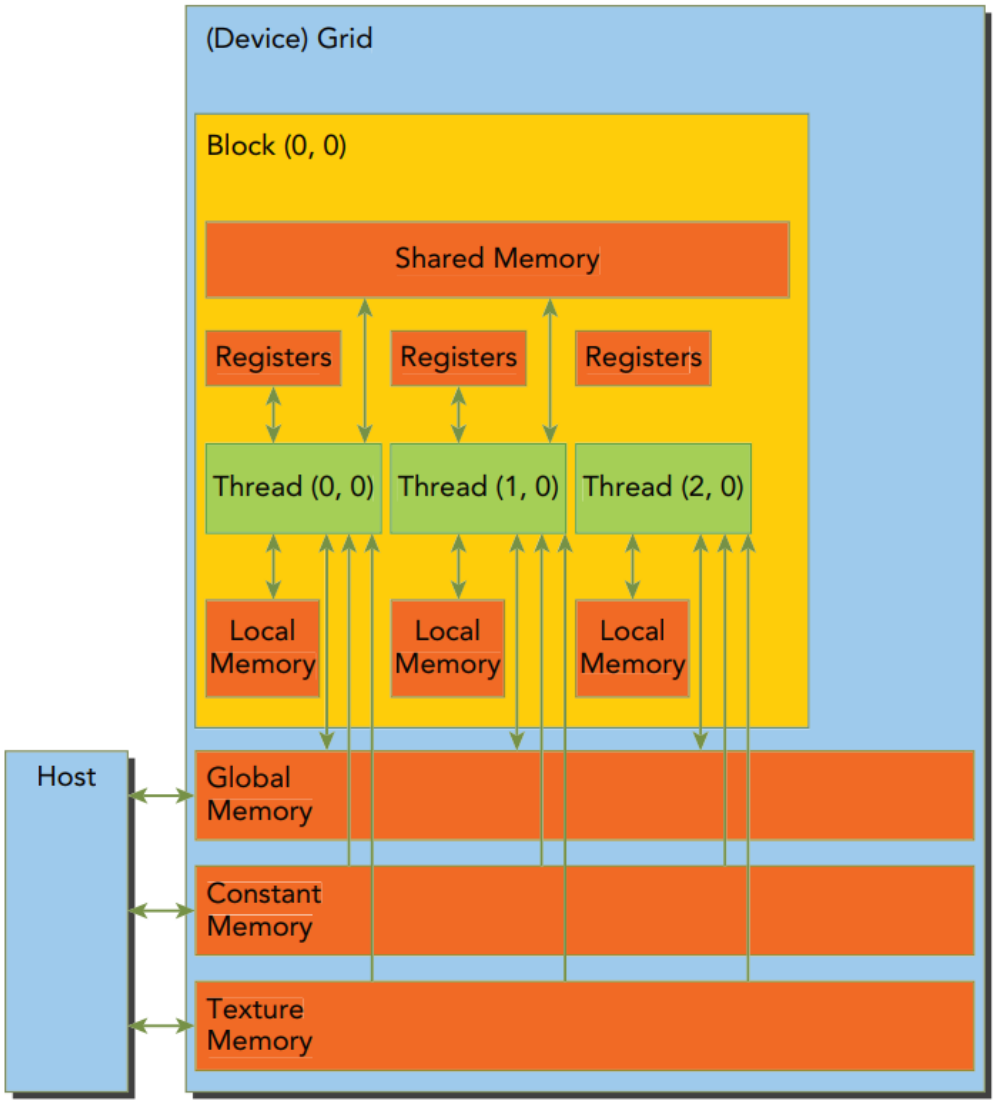
- Non-programmable memory: L1 cache and L2 cache

CUDA memory model

exposes many types of programmable memory:

- Registers
- Shared memory
- Local memory
- Constant memory
- Texture memory
- Global memory.

CUDA Memory Model



- A thread in a kernel has its own private local memory.
- A thread block has its own shared memory.
- All threads can access global memory.
- Two read-only memory accessible by all threads: constant and texture memory spaces.

Registers

- **Registers are the fastest memory space on a GPU.**
  - An automatic variable declared in a kernel without any other type qualifiers is generally stored in a register.
- **Register variables are private to each thread.**
- **Register variables share their lifetime with the kernel.**
  - Once a kernel completes execution, a register variable cannot be accessed again.
- **Registers are scarce resources that are partitioned** among active warps in an SM.

Registers

- On Fermi GPUs, there is a **hardware limit of 63 registers per thread**.
  - If a kernel uses more registers than the hardware limit, the excess registers will spill over to local memory.
- This **register spilling** can have **adverse** performance consequences.
- The nvcc compiler uses **heuristics** to minimize register usage and avoid register spilling.
- Another approach:

```
__global__ void
__launch_bounds__(maxThreadsPerBlock, minBlocksPerMultiprocessor)
        kernel(...) {
        // your kernel body
}
```

specifies the maximum number of threads per block that a kernel will launch

Local Memory

- Variables in a kernel that are **eligible for registers** but **cannot fit into the register** space allocated for that kernel will **spill into local memory**.

- **Variables that the compiler is likely to place in local memory are:**
  - Local arrays referenced with indices whose values cannot be determined at compile-time.
  - Large local structures or arrays that would consume too much register space.
  - Any variable that does not fit within the kernel register limit.

Local Memory

- **The name "local memory" is misleading**

- Values spilled to local memory **reside in the same physical location as global memory**
  - local memory accesses are characterized by high latency and low bandwidth

- For GPUs with compute capability 2.0 and higher, local memory data is also **cached in a per-SM L1 and per-device L2 cache.**

Shared Memory

- Variables decorated with the **following attribute** in a kernel are stored in shared memory:

```
__shared__
```

- Because **shared memory is on-chip**, it has a much **higher bandwidth** and much **lower latency** than local or global memory.

- It is used similarly to CPU L1 cache, but is also **programmable**.

- Each SM has a **limited amount** of shared memory that is partitioned among **thread blocks**.

Shared Memory

- Shared memory is declared in the scope of a kernel function but **shares its lifetime with a thread block**.

- When a thread block is **finished executing**, its allocation of shared memory will be **released and assigned** to other thread blocks.

- Shared memory serves as a basic means for **inter-thread communication.**

- Threads within a block can **cooperate by sharing data stored in shared memory**.

Shared Memory

- Access to shared memory must be **synchronized** using the following CUDA runtime call:

```
void __syncthreads();
```

- This function creates a **barrier** which all threads in the same thread block must reach **before any other thread is allowed to proceed.**

- By creating a barrier for all threads within a thread block, you can **prevent a potential data hazard**.

Shared Memory

- The L1 cache and shared memory for an SM use the same **64 KB of on-chip memory**,
  - statically partitioned but can be dynamically configured at runtime by:

```
cudaError_t cudaFuncSetCacheConfig(const void* func,
enum cudaFuncCache cacheConfig);
```

- This function configures the partitioning of on-chip memory on a **per-kernel basis**.

- The supported cache **configurations** are:

cudaFuncCachePreferNone: no preference (default)

cudaFuncCachePreferShared: prefer 48KB shared memory and 16KB L1 cache

cudaFuncCachePreferL1: prefer 48KB L1 cache and 16KB shared memory

cudaFuncCachePreferEqual: Prefer equal size of L1 cache and shared memory, both 32KB

Constant Memory

- Constant memory resides in **device memory** and is cached in a dedicated, per-SM constant cache.

- A constant variable is decorated with the following attribute: `__constant__`

- Constant variables must be declared with **global scope**, outside of any kernels.

- A **limited amount** of constant memory can be declared
  - 64 KB for all compute capabilities.

Constant Memory

- Constant memory is **statically declared** and **visible to all kernels** in the same compilation unit
  - Kernels can **only read from** constant memory.

- Constant memory must be initialized by host using:

```
cudaError_t cudaMemcpyToSymbol(const void* symbol,
const void* src, size_t count);
```

- Constant memory performs best when **all threads in a warp read from the same memory address**.
  - E.g., a coefficient for a mathematical formula

Texture Memory

- Texture memory **resides in device memory** and is cached in a per-SM, read-only cache.
- Texture memory is a type of global memory that is accessed **through a dedicated read-only cache**.
- Texture memory is optimized for **2D spatial locality**
  - Threads in a warp that use texture memory to access **2D data** will achieve the **best performance**.

Global Memory

- Global memory is the **largest, highest-latency**, and most commonly used memory on a GPU.

- A variable in global memory can be declared **statically** or **dynamically**.
  - **Statically**: in device code using `__device__`
  - **Dynamically**: using `cudaMalloc` and freed by `cudaFree`.

- **Potential hazard** of multiple threads in different blocks **concurrently modifying** the **same location** in global memory.
  - Thread execution cannot be synchronized across thread blocks

Global Memory

- Global memory resides in **device memory** and is accessible via **32-byte, 64-byte, or 128-byte** memory transactions.
- Optimizing memory transactions are vital to obtaining **optimal performance**.
- When a warp performs a memory load/store, the **number of transactions** required to satisfy that request typically depends on:
  - Distribution of memory addresses across the threads of that warp.
  - Alignment of memory addresses per transaction.

GPU Caches

- Like CPU caches, **GPU caches** are **non-programmable** memory.

- There are **four types of cache** in GPU devices:
  - L1: shared by all SMs, store data in local and global memory
  - L2: shared by all SMs, store data in local and global memory
  - Read-only constant: for constant memory
  - Read-only texture: for texture memory

- On CPU, **both memory loads** and **stores can be cached**.

- On GPU, **only memory load** operations can **be cached**; **memory store** operations **cannot be cached**.

CUDA Variable Declaration Summary

QUALIFIER	VARIABLE NAME	MEMORY	SCOPE	LIFESPAN
	`float var`	Register	Thread	Thread
	`float var[100]`	Local	Thread	Thread
`__shared__`	`float var†`	Shared	Block	Block
`__device__`	`float var†`	Global	Global	Application
`__constant__`	`float var†`	Constant	Global	Application

Salient Features of Device Memory

MEMORY	ON/OFF CHIP	CACHED	ACCESS	SCOPE	LIFETIME
Register	On	n/a	R/W	1 thread	Thread
Local	Off	†	R/W	1 thread	Thread
Shared	On	n/a	R/W	All threads in block	Block
Global	Off	†	R/W	All threads + host	Host allocation
Constant	Off	Yes	R	All threads + host	Host allocation
Texture	Off	Yes	R	All threads + host	Host allocation

† Cached only on devices of compute capability 2.x

2. Memory Management

2.1 Memory Allocation and Deallocation

Memory Allocation and Deallocation

- Kernel functions operate in the device memory space
  - CUDA runtime provides functions to allocate and deallocate device memory.

- Allocate global memory on the host using:

```
cudaError_t cudaMalloc(void **devPtr, size_t count);
```

- Initialize global memory using:

```
cudaError_t cudaMemset(void *devPtr, int value,
size_t count);
```

- Deallocated using:

```
cudaError_t cudaFree(void *devPtr);
```

2.2 Memory Transfer

Memory Transfer

- Once global memory is allocated, you can transfer data to the device from the host using:

```
cudaError_t cudaMemcpy(void *dst, const void
*src, size_t count, enum cudaMemcpyKind kind);
```

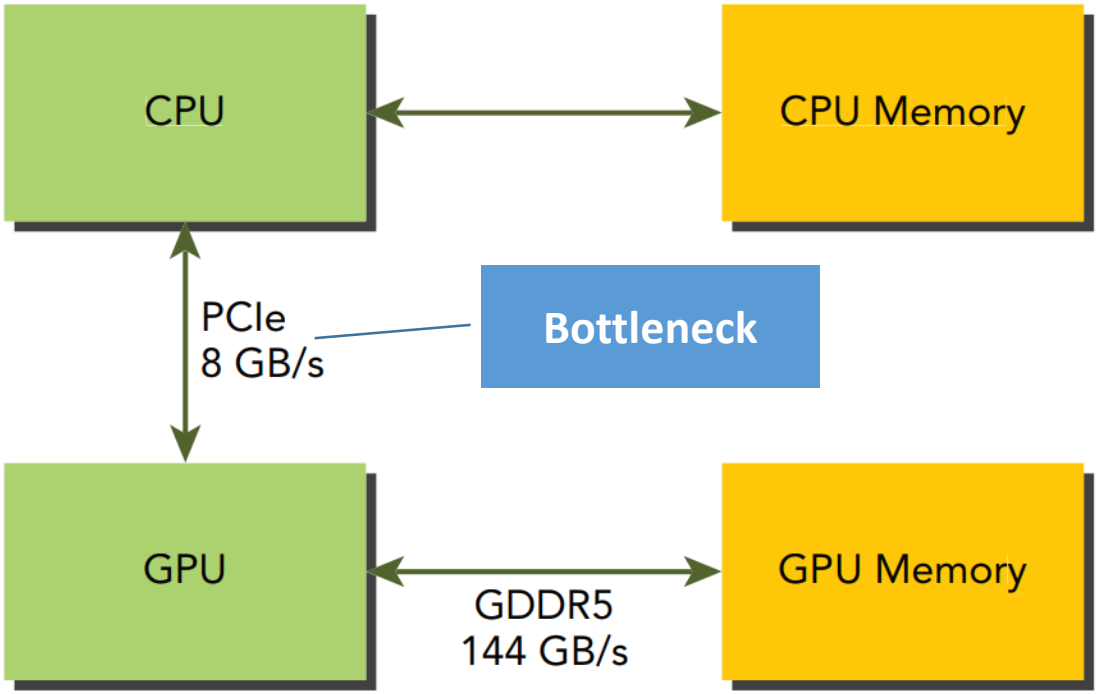
- The variable kind specifies the direction of the copy:
  - `cudaMemcpyHostToHost`
  - `cudaMemcpyHostToDevice`
  - `cudaMemcpyDeviceToHost`
  - `cudaMemcpyDeviceToDevice`



memTransfer.cu

Connectivity of CPU and GPU Memory

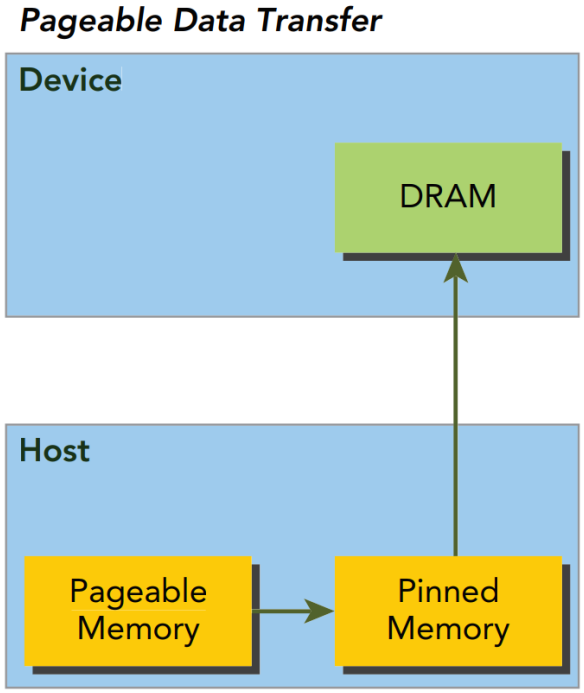
- Data transfers **between the host and device** can **throttle** overall application performance
  - You should always be thinking of ways to minimize host-device transfers



2.3 Pinned Memory

Pageable Data Transfer

- **Allocated host memory is by default pageable**
  - subject to page fault operations
  - move data in host virtual memory to different physical locations.
- **GPU cannot safely access data in pageable host memory**
  - no control over when the host OS may choose to physically move that data.



页锁定内存(Page-locked Memory)

或称固定内存(Pinned Memory)

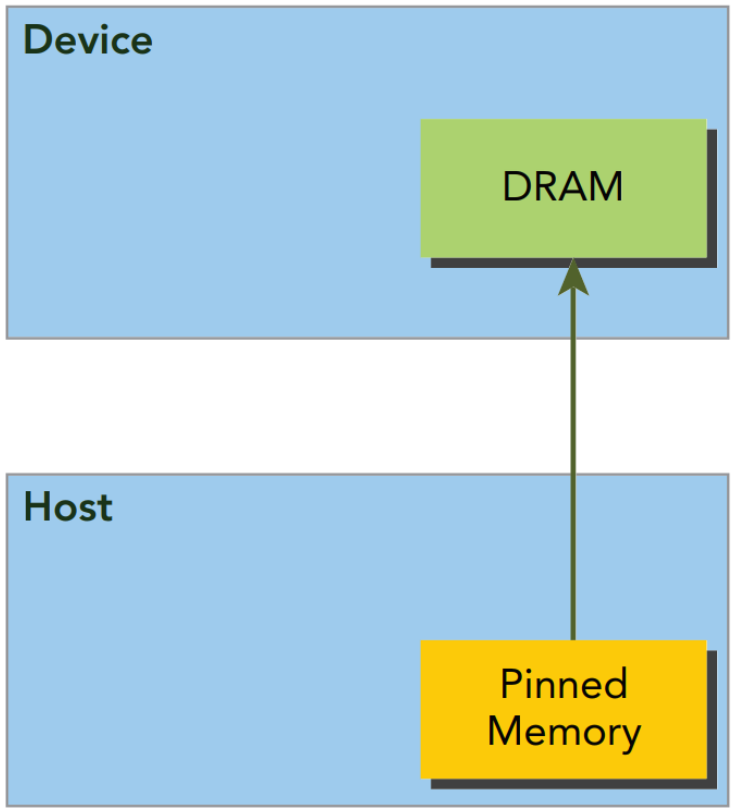
When transferring data from pageable host memory to device memory:

- the CUDA driver first allocates temporary **page-locked or pinned host memory**,
- copies the source host data to pinned memory,
- transfers the data from pinned memory to device memory

Pinned Data Transfer

- The CUDA runtime allows to directly allocate **pinned host memory** using:

```
cudaError_t cudaMallocHost(void **devPtr, size_t count);
```



- Pinned memory can be accessed directly by the device
- it can be read and written with much **higher** bandwidth than pageable memory.

Memory Transfer Between Host and Device

- **Pinned memory** is **more expensive to allocate and deallocate** than pageable memory, but it provides **higher transfer throughput** for large data transfers.
- The **speedup** achieved when using pinned memory depends on **device compute capability**.
- **Batching many small transfers** into one larger transfer **improves performance** because it reduces per-transfer overhead.
- **Data transfers** between the host and device can sometimes be **overlapped** with **kernel execution**.

2.4 Zero-Copy Memory

Zero-Copy Memory

- In general, the host cannot directly access device variables, and the device cannot directly access host variables.

- There is **one exception** to this rule: **zero-copy memory.**
  - Both the host and device can access zero-copy memory.

- **Advantages**:
  - Leveraging host memory when there is insufficient device memory
  - Avoiding **explicit** data transfer between host and device
  - Improving PCIe transfer rates

Zero-Copy Memory

- **Zero-copy memory** is **pinned (non-pageable) memory** that is **mapped into the device address space**.

- Create a mapped, pinned memory region with **`cudaHostAlloc`** and free with **`cudaFreeHost:`**

```
cudaHostAlloc(void **pHost, size_t count, unsigned int flags);
```

- **`flags`** parameter:
  - **`cudaHostAllocDefault`** — makes the behavior of cudaHostAlloc identical to cudaMallocHost
  - **`cudaHostAllocPortable`** — returns pinned memory that can be used by all CUDA contexts
  - **`cudaHostAllocWriteCombined`** — a good option for buffers that will be written by host and read by device
  - **`cudaHostAllocMapped`** — returns host memory that is mapped into the device address space

The most relevant flag to zero-copy memory

Zero-Copy Memory

- **Obtain device pointer** for mapped pinned memory:

```
cudaHostGetDevicePointer(void **pDevice, void *pHost, unsigned int flags);
```

a device pointer that can be referenced on the device to access mapped, pinned host memory

- Using **zero-copy memory** as a supplement to device memory with **frequent read/write operations** will significantly slow performance.
  - Every memory transaction to mapped memory **must pass over the PCIe bus**

Zero-Copy Memory

```
$ nvprof ./sumZerocpy
Using Device 0: Tesla M2090 Vector size 1024 power 10  nbytes    4 KB
Time(%)      Time     Calls       Avg       Min       Max  Name
 27.18%  3.7760us         1  3.7760us  3.7760us  3.7760us  sumArraysZeroCopy
 11.80%  1.6390us         1  1.6390us  1.6390us  1.6390us  sumArrays
 25.56%  3.5520us         3  1.1840us  1.0240us  1.5040us  [CUDA memcpy HtoD]
 35.47%  4.9280us         2  2.4640us  2.4640us  2.4640us  [CUDA memcpy DtoH]
```

the elapsed time of the kernel reading from **zero-copy memory** is **2.31 times slower** than the kernel using only **device memory**.



sumArrayZerocpy.cu

Zero-copy Memory vs Device Memory

- Check performance for different array sizes

SIZE	DEVICE MEMORY (ELAPSED TIME)	ZERO-COPY MEMORY (ELAPSED TIME)	SLOWDOWN
1 K	1.5820 us	2.9150 us	1.84
4 K	1.6640 us	3.7900 us	2.28
16 K	1.6740 us	7.4570 us	4.45
64 K	2.3910 us	22.586 us	9.45
256 K	7.2890 us	82.733 us	11.35
1 M	28.267 us	321.57 us	11.38
4 M	104.17 us	1.2741 ms	12.23
16 M	408.03 us	5.0903 ms	12.47
64 M	1.6276 ms	20.347 ms	12.50

- if you share a small amount of data between the host and device, zero-copy memory may be a good choice because it simplifies programming and offers reasonable performance.
- For larger datasets with discrete GPUs connected via the PCIe bus, zero-copy memory is a poor choice and causes significant performance degradation.

Zero-Copy Memory

- **Integrated GPU**:
  - CPUs and GPUs are fused onto a single die and physically share main memory.
  - Zero-copy memory is more likely to benefit both performance and programmability because no copies over the PCIe bus are necessary.

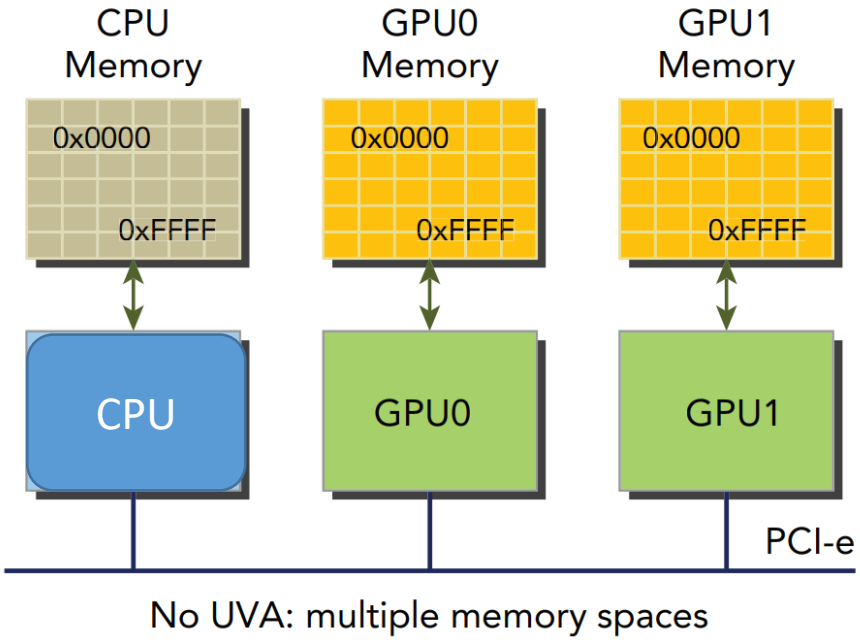
- **Discrete GPU:**
  - zero-copy memory is advantageous only in special cases.

- **Be careful to not overuse zero-copy memory**.
  - Device kernels that read from zero-copy memory can be very slow due to its high-latency

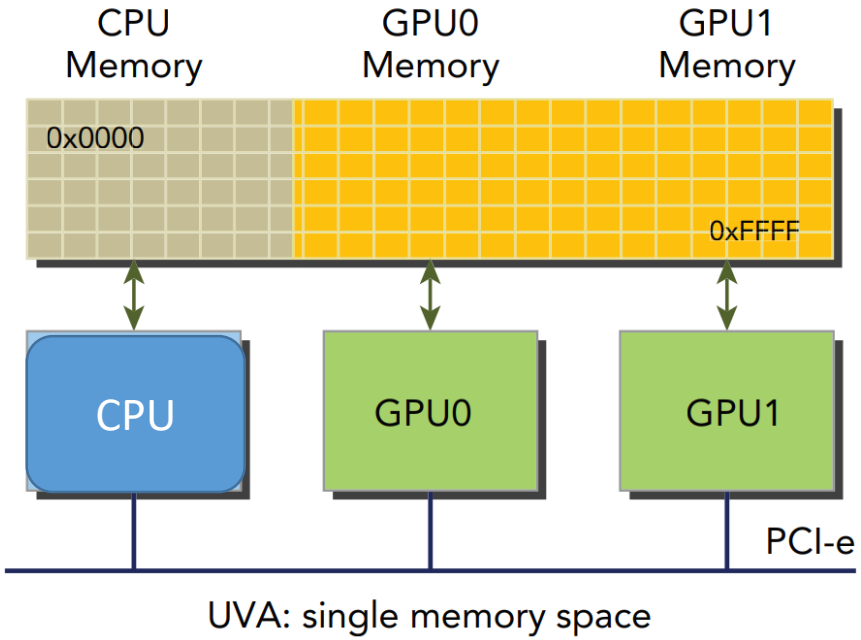
2.5 Unified Virtual Addressing

Unified Virtual Addressing

Devices with **compute capability 2.0** and later support a special addressing mode called **Unified Virtual Addressing (UVA).**



No UVA: multiple memory spaces



UVA: single memory space

No UVA: need to manage which pointers referred to host memory and which referred to device memory.

UVA: host memory and device memory share a single virtual address space

Unified Virtual Addressing

- **Recall the zero-copy example**

① Allocated mapped, pinned host memory.
② ~~Acquired the device pointer to the mapped, pinned memory using a CUDA runtime function.~~
③ Passed the device pointer to your kernel.

With UVA, there is no need to acquire the device pointer or manage two pointers to what is physically the same data.

2.6 Unified Memory

Unified Memory

- Unified Memory creates a **pool of managed memory**
  - Each allocation from this memory pool is accessible on both CPU and GPU with the same memory address
- The underlying system **automatically migrates** data in unified memory **between host and device**.
- Data movement is **transparent to the application**
  - Greatly simplifying the application code.

Unified Memory vs. UVA

- **UVA:** not automatically migrate data from one physical location to another
  - that is a capability unique to Unified Memory.

- **Unified Memory:** transparently migrate data on demand to the host or device
  - offers a "single-pointer-to-data" model that is conceptually similar to **zero-copy memory**.
  - However, zero-copy memory is allocated in host memory
  - Zero-copy memory suffers from high-latency accesses over the PCIe bus.

Managed Memory

- Two types of memory in a kernel:
  - **managed memory** that is controlled by the system
  - **un-managed memory** that must be explicitly allocated and transferred by the application.

- **Managed memory** refers to Unified Memory allocations that are automatically managed by the underlying system

- Managed memory can be allocated **statically or dynamically**

```
__device__ __managed__ int y;  //statically

cudaMallocManaged(void **devPtr, size_t size,
    unsigned int flags=0)  //dynamically
```

3. Memory Access Patterns

Memory Access Patterns

- **Maximizing use of global memory bandwidth** is a fundamental step in kernel performance tuning.

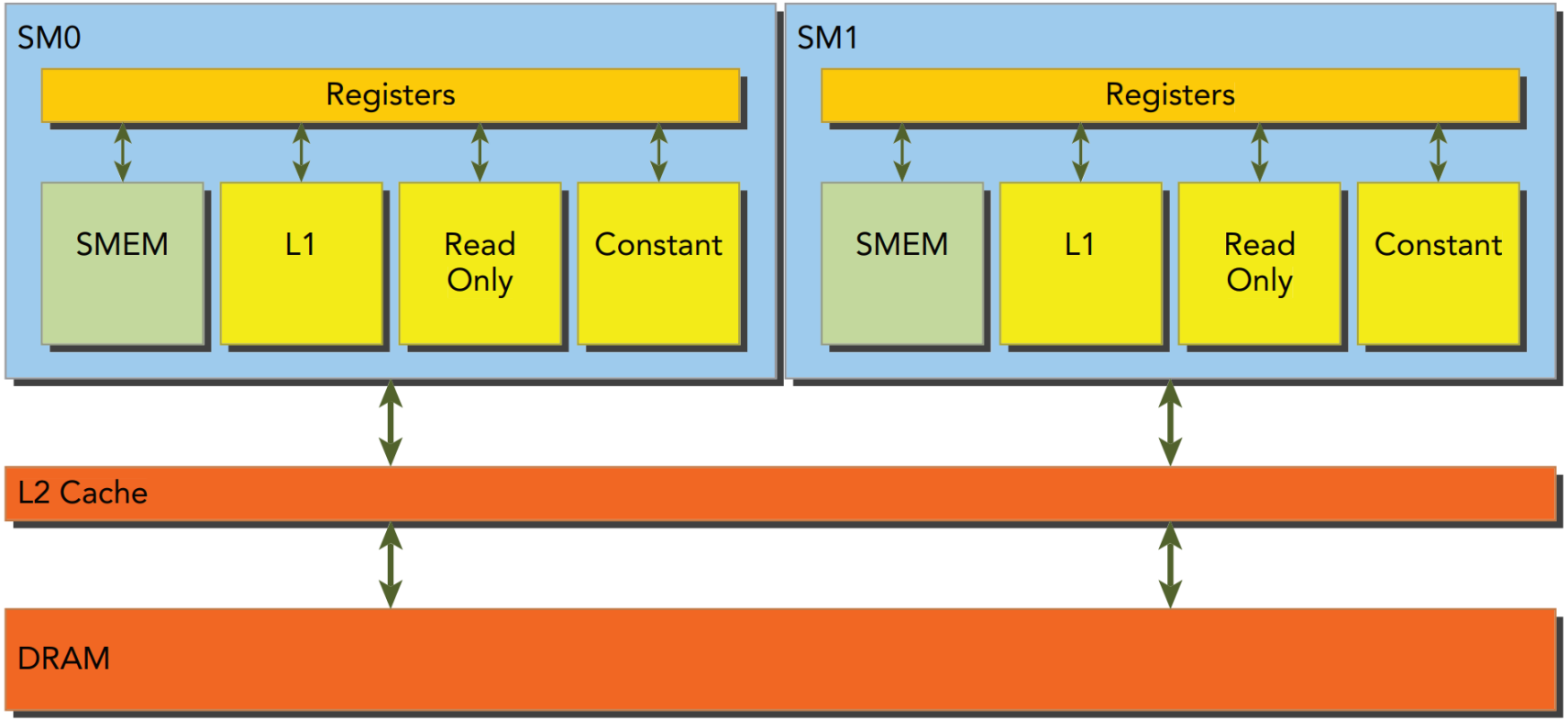
- One of the **distinguishing features** of the CUDA execution model is that **instructions are issued and executed per warp**.
  - Memory operations are also issued per warp.

- Depending on the distribution of memory addresses within a warp, **memory accesses can be classified** into **different patterns**.

3.1 Aligned and Coalesced Access
(对齐和合并访问)

Aligned and Coalesced Access

- Global memory loads/stores are staged through caches



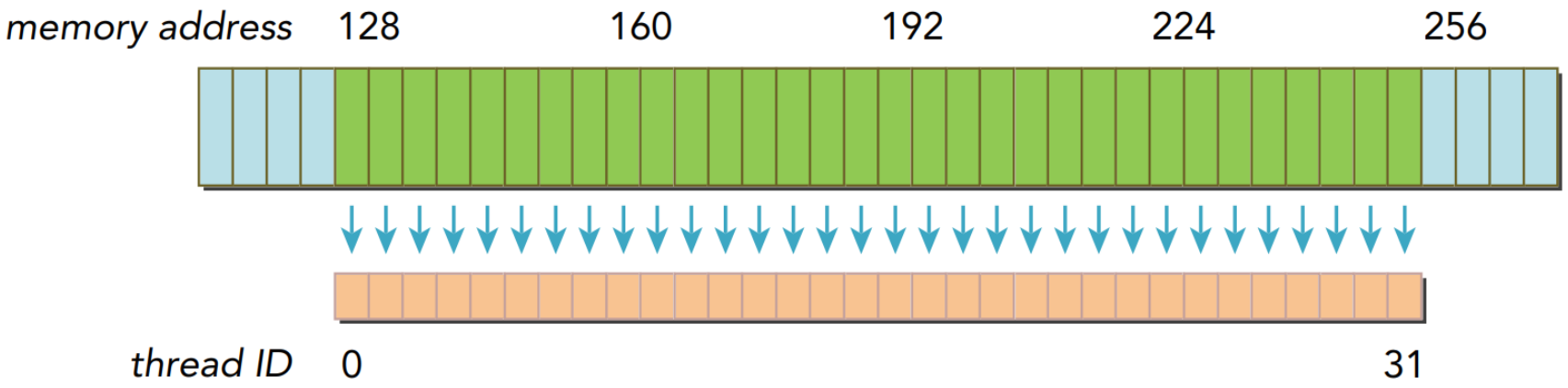
Types of Device Memory Accesses

- **Aligned memory accesses（对齐的内存访问）：**
  - when the first address of a device memory transaction is an even multiple of the cache granularity being used to service the transaction
    - either 32 bytes for L2 cache or 128 bytes for L1 cache.
  - Performing a misaligned load will cause wasted bandwidth.

- **Coalesced memory accesses （合并的内存访问）：**
  - when all 32 threads in a warp access a contiguous chunk of memory.

Aligned Coalesced Memory Access

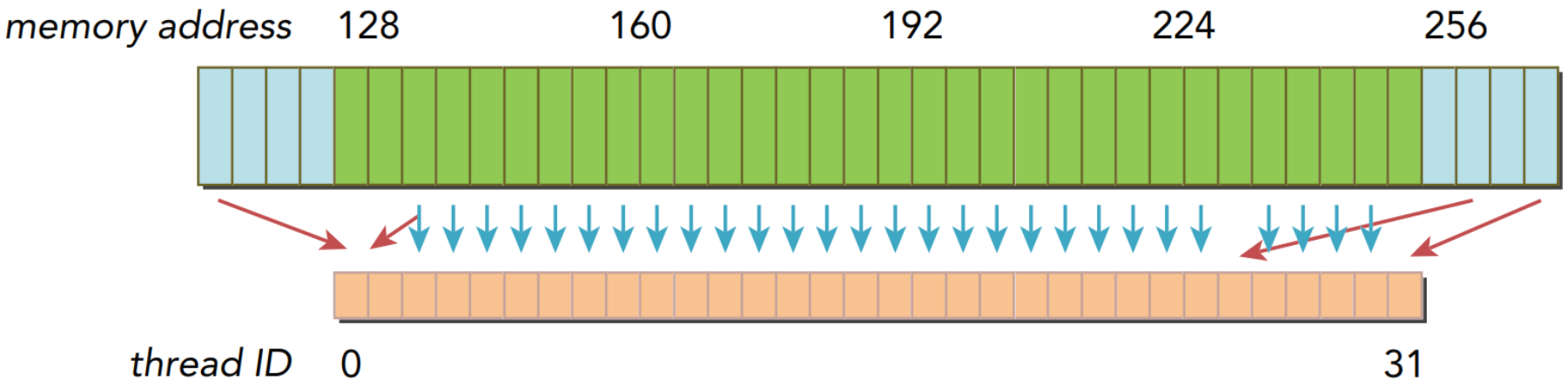
- **Ideal case**
  - A wrap accessing a **contiguous chunk** of memory starting at an **aligned memory address**.



only a single 128-byte memory transaction is required to read the data from device memory

Misaligned Uncoalesced Memory Access

- **Bad case:** leading to wasted bandwidth



Three 128-byte memory transactions required to read the data from device memory:

- one starting at offset 0 to include the data being read below the contiguous region,
- one at offset 256 to read the data being read above the contiguous region
- one at offset 128 that fetches the bulk of the data.

3.2 Global Memory Reads

Global Memory Reads

- Whether global memory load operations **pass through the L1 cache** depends on two factors:
  - **Device compute capability**
  - **Compiler options**

    -Xptxas -dlcm=cg

- With the L1 cache **disabled**, all load requests to global memory go directly to the **L2 cache**

- When an **L2 miss occurs**, the requests are **serviced by DRAM**.

Memory Load Access Patterns

- **Two types of memory loads**:
  - **Cached load** (L1 cache enabled)
  - **Uncached load** (L1 cache disabled)

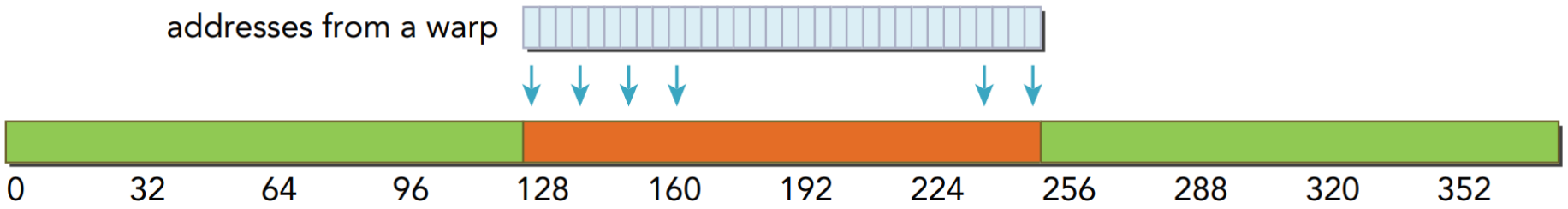
- The access pattern for memory loads can be characterized by the following **combinations**:
  - **Cached versus uncached**: The load is cached if L1 cache is enabled
  - **Aligned versus misaligned**: The load is aligned if the first address of a memory access is a multiple of 32 bytes
  - **Coalesced versus uncoalesced**: The load is coalesced if a warp accesses a contiguous chunk of data

Cached Load

- Cached load operations **pass through L1 cache** and are serviced by device memory transactions at the granularity of an L1 cache line, **128-bytes**.

- Cached loads can be classified as **aligned/misaligned** and **coalesced/uncoalesced**.

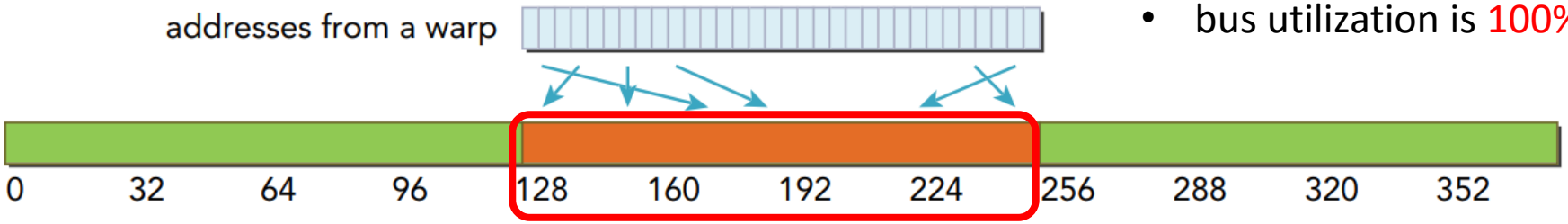
ideal case: aligned and coalesced memory accesses.



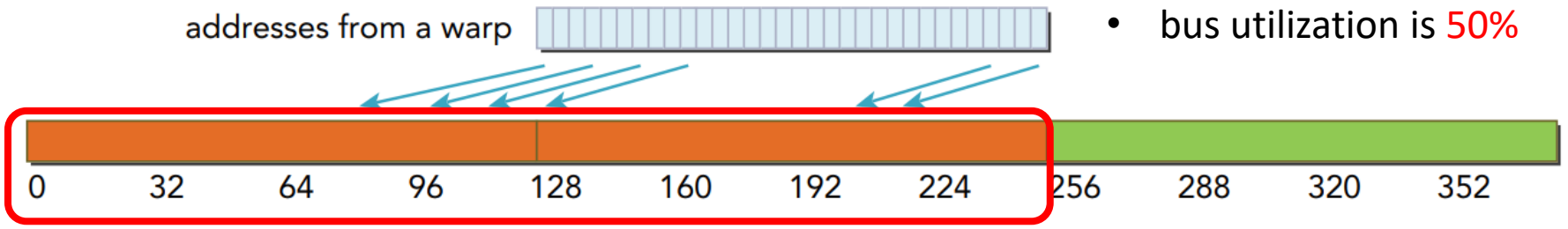
- The addresses requested by all threads in a warp fall within one cache line of 128 bytes.
- Only a single 128-byte transaction is required to complete the memory load operation.
- bus utilization is 100%

Cached Load

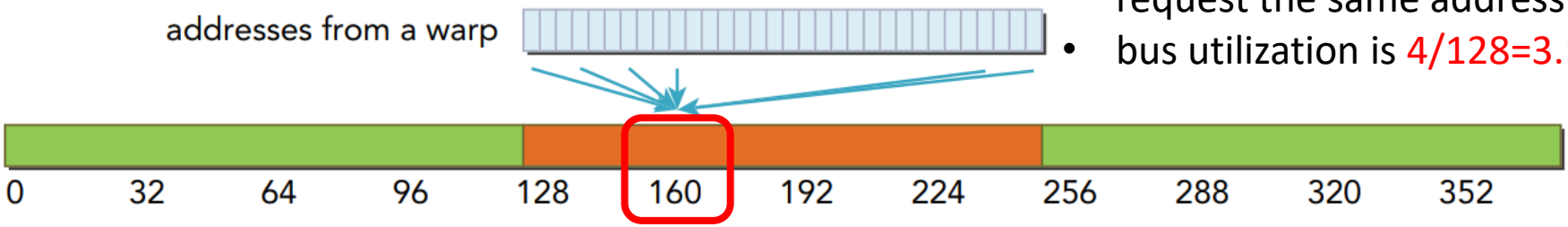
- access is aligned and referenced addresses are not consecutive
- bus utilization is 100%



- addresses requested fall across two segments
- bus utilization is 50%

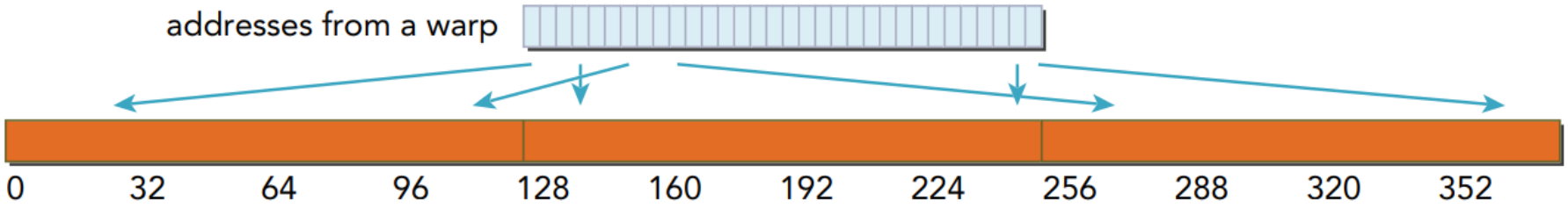


- all threads in the warp request the same address
- bus utilization is 4/128=3.1%



Cached Load

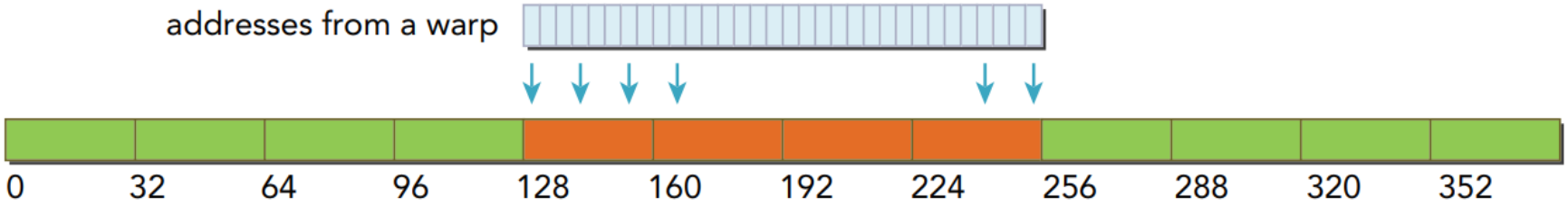
- **Worst Case:**
  - threads in a warp request 32 four-byte addresses scattered across global memory.



- **CPU L1 Cache vs. GPU L1 Cache**
  - **CPU L1 Cache:** optimized for both spatial and temporal locality
  - **GPU L1 Cache**: designed for spatial but not temporal locality

Uncached Loads

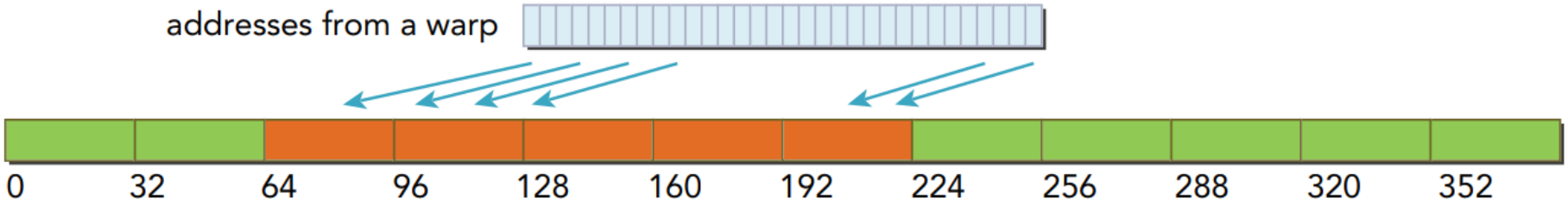
- Uncached loads do **not pass** through the **L1 cache**
  - performed at the **granularity** of memory segments (32-bytes) and not cache lines (128-bytes).
  - more fine-grained loads
  - better bus utilization for misaligned or uncoalesced memory accesses.



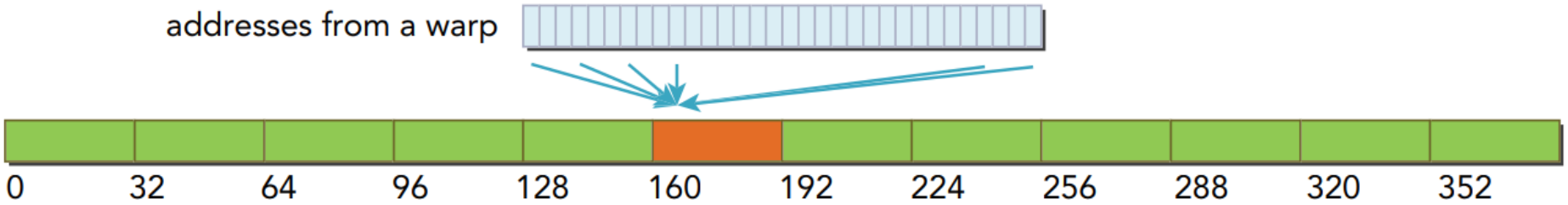
- addresses for the 128 bytes requested fall within **4 segments**
- bus utilization is **100%**

Uncached Loads

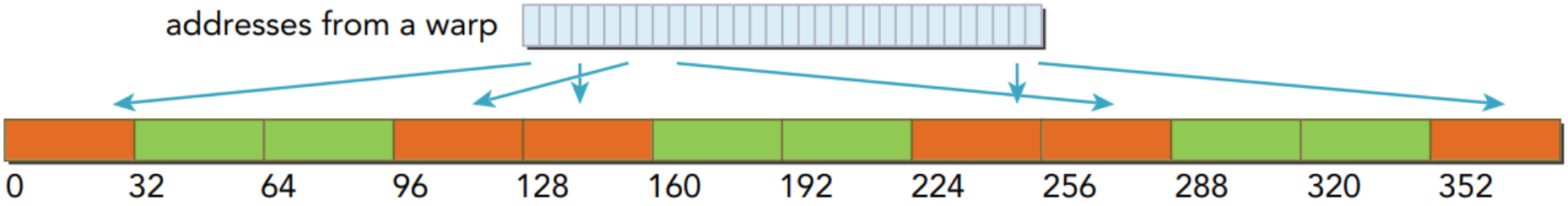
- a warp requests 32 consecutive 4-byte elements.
- addresses fall within at most 5 segments
- bus utilization is at least 80%.
- Performance is improved with uncached loads



- all threads in the warp request the same data.
- bus utilization is 4/32 = 12.5%



Worst case: a warp requests 32 4-byte words scattered across global memory



Example of Misaligned Reads

- new index k is shifted up by a given offset, which might cause misaligned loads

```
__global__ void readOffset(float *A, float *B, float *C, const int n,
    int offset) {
   unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
   unsigned int k = i + offset;
   if (k < n) C[i] = A[k] + B[k];
}
```

Misaligned offset

```
$ ./readSegment 0
readOffset <<< 32768,   512 >>> offset     0 elapsed 0.001820 sec
$ ./readSegment 11
readOffset <<< 32768,   512 >>> offset    11 elapsed 0.001949 sec
$ ./readSegment 128
readOffset <<< 32768,   512 >>> offset   128 elapsed 0.001821 sec
```

Read-Only Cache

- Read-only cache can support global memory loads
  - The granularity is 32 bytes
  - better for scattered reads than the L1 cache.

- Two ways to direct memory reads through the read-only cache:
  - Using the function `__ldg`
  - Using a declaration qualifier on the pointer being dereferenced

Read-Only Cache

```
__global__ void copyKernel(int *out, int *in) {
   int idx  = blockIdx.x * blockDim.x + threadIdx.x;
   out[idx] = in[idx];
}
```



```
__global__ void copyKernel(int *out, int *in) {
   int idx  = blockIdx.x * blockDim.x + threadIdx.x;
   out[idx] = __ldg(&in[idx]);
}
```

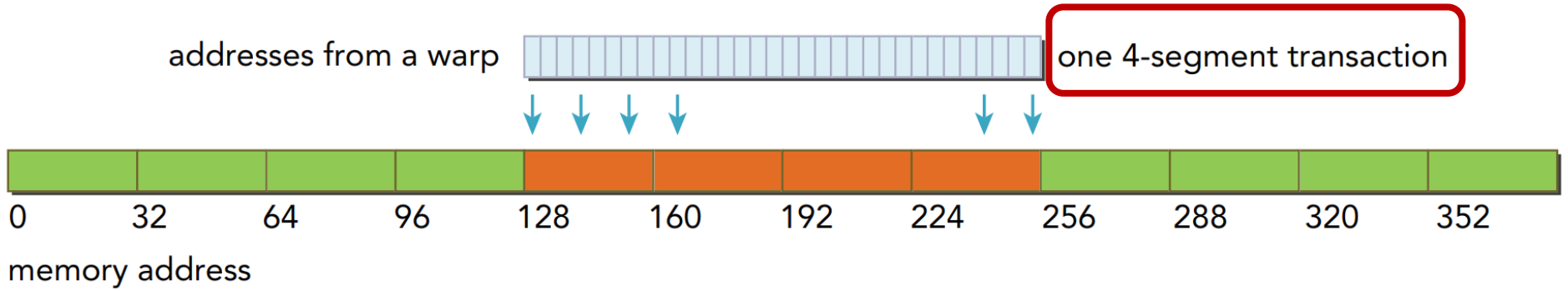
```
__global__ void copyKernel(int * __restrict__ out,
     const      int * __restrict__ in) {
   int idx  = blockIdx.x * blockDim.x + threadIdx.x;
   out[idx] = in[idx];
}
```

3.3 Global Memory Writes

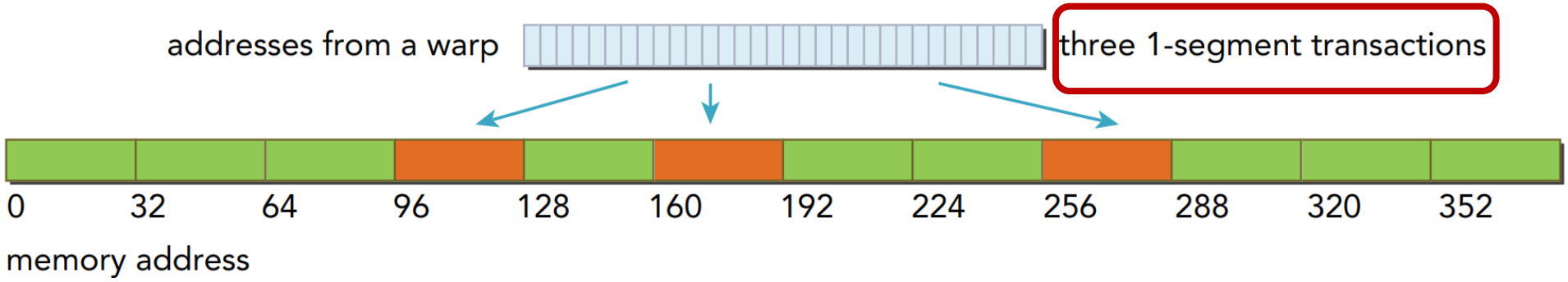
Global Memory Writes

- **L1 cache** is not used for store operations
- **Store operations** are only cached in the **L2 cache**
- Stores are performed at a 32-byte segment **granularity**.
- Memory transactions can be one, two, or four segments at a time.

Global Memory Writes

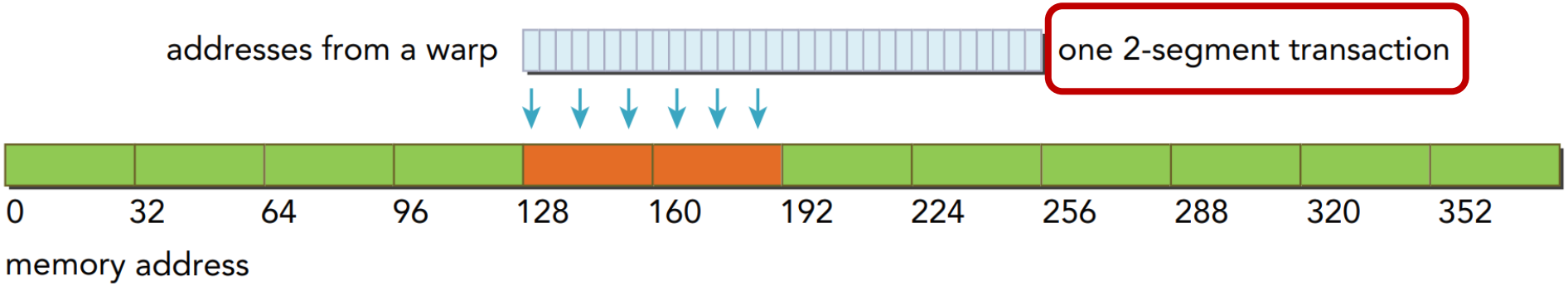


memory access is aligned and all threads in a warp access a consecutive 128-byte range



memory access is aligned, but the addresses are scattered along a 192-byte range.

Global Memory Writes



memory access is aligned and the addresses accessed are in a consecutive 64-byte range

Example of Misaligned Writes

- Index k is shifted by a given offset
  - writing to array C uses the offset index k, likely causing misaligned writes

```
__global__ void writeOffset(float *A, float *B, float *C,
      const int n, int offset) {
   unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
   unsigned int k = i + offset;
   if (k < n) C[k] = A[i] + B[i];
}
```

Misaligned writes

```
$ ./writeSegment 0
writeOffset <<< 2048,  512 >>> offset    0 elapsed 0.000134 sec
$ ./writeSegment 11
writeOffset <<< 2048,  512 >>> offset   11 elapsed 0.000184 sec
$ ./writeSegment 128
writeOffset <<< 2048,  512 >>> offset  128 elapsed 0.000134 sec
```

3.4 Array of Structures versus Structure of Arrays

Array of Structures vs. Structure of Arrays

- **Array of Structures (AoS)**

```
struct innerStruct {
    float x;
    float y;
};

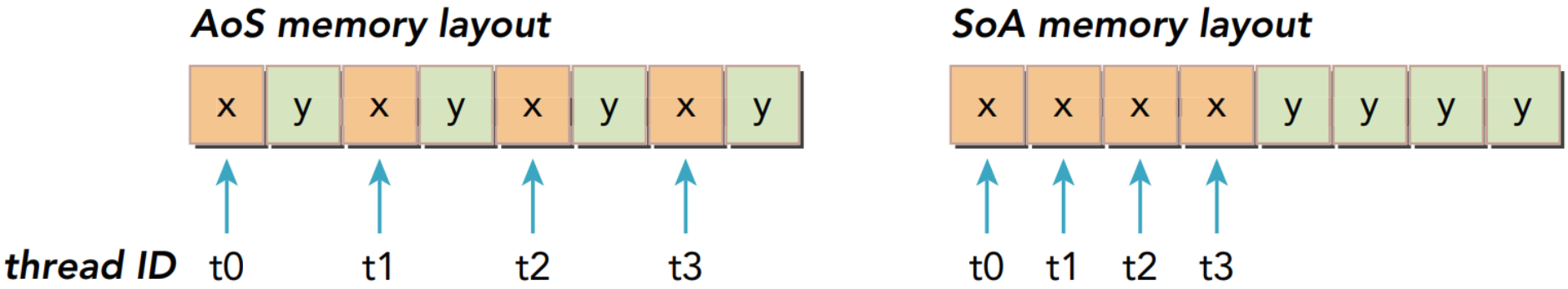
struct innerStruct myAoS[N];
```

- **Structure of Arrays (SoA)**

```
struct innerArray {
    float x[N];
    float y[N];
};

struct innerArray moa;
```

Memory Layout



- Storing the data in SoA fashion makes **full use of GPU memory bandwidth**.
- In CUDA C programming, SoA is typically preferred because data elements are pre-arranged for efficient **coalesced access** to global memory

Example: AoS

- Example: Simple Math with the AoS Data Layout

```
__global__ void testInnerStruct(innerStruct *data,
    innerStruct *result, const int n) {
   unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;

   if (i < n) {
      innerStruct tmp = data[i];
      tmp.x += 10.f;
      tmp.y += 20.f;
      result[i] = tmp;
   }
}
```



simpleMathAoS.cu

```
gld_efficiency  50.00%
gst_efficiency  50.00%
```

```
// kernel 2: testInnerStruct
iStart = seconds();
testInnerStruct <<< grid, block >>> (d_A, d_C, nElem);
cudaDeviceSynchronize();
iElaps = seconds() - iStart;
printf("innerstruct <<< %3d, %3d >>> elapsed %f sec\n",grid.x,
   block.x,iElaps);
cudaMemcpy(gpuRef, d_C, nBytes, cudaMemcpyDeviceToHost);
checkInnerStruct(hostRef, gpuRef, nElem);
```

Example: SoA

- Example: Simple Math with the SoA Data Layout

```
__global__ void testInnerArray(InnerArray *data,
    InnerArray *result, const int n) {
   unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;

   if (i<n) {
      float tmpx = data->x[i];
      float tmpy = data->y[i];

      tmpx += 10.f;
      tmpy += 20.f;
      result->x[i] = tmpx;
      result->y[i] = tmpy;
   }
}
```

```
gld_efficiency  100.00%
gst_efficiency  100.00%
```

3.5 Performance Tuning

Performance Tuning

- Two goals when optimizing device mem BW utilization:
  - Aligned and coalesced memory accesses that reduce wasted bandwidth
  - Sufficient concurrent memory operations to hide memory latency

- **Unrolling Techniques**
  - Unrolling loops that contain memory operations adds more independent, memory operations to the pipeline.

- **Exposing More Parallelism**
  - To expose sufficient parallelism, find the best execution configuration for that kernel.

Maximizing Bandwidth Utilization

- Efficient use of bytes moving between device DRAM and SM on-chip memory:
  - memory access patterns should be aligned and coalesced.

- # of memory operations concurrently in-flight:
  - Maximizing the number of in-flight memory operations is possible through:
    - 1) **unrolling**, yielding more independent memory accesses per thread,
    - 2) **modifying the execution configuration of a kernel** launch to expose more parallelism to each SM.

4. What Bandwidth Can a Kernel Achieve?

4.1 Memory Bandwidth

Memory Bandwidth

- **Theoretical bandwidth**
  - the absolute maximum bandwidth achievable with the hardware at hand.

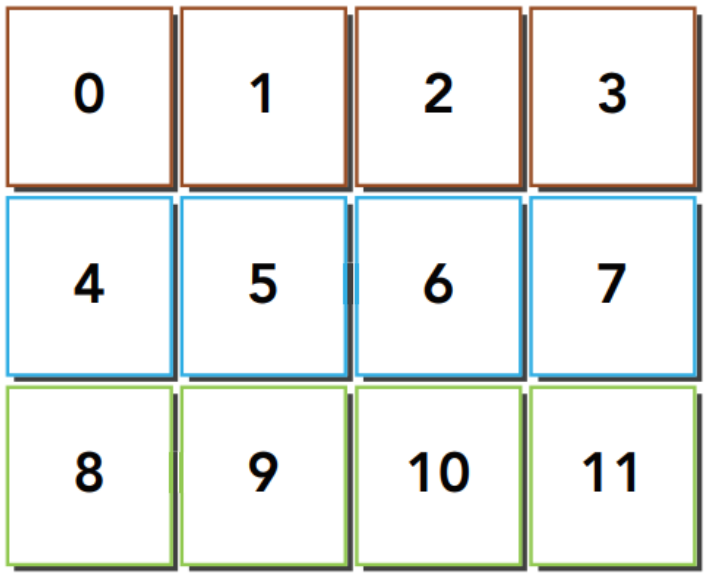
- **Effective bandwidth**
  - the measured bandwidth that a kernel actually achieves

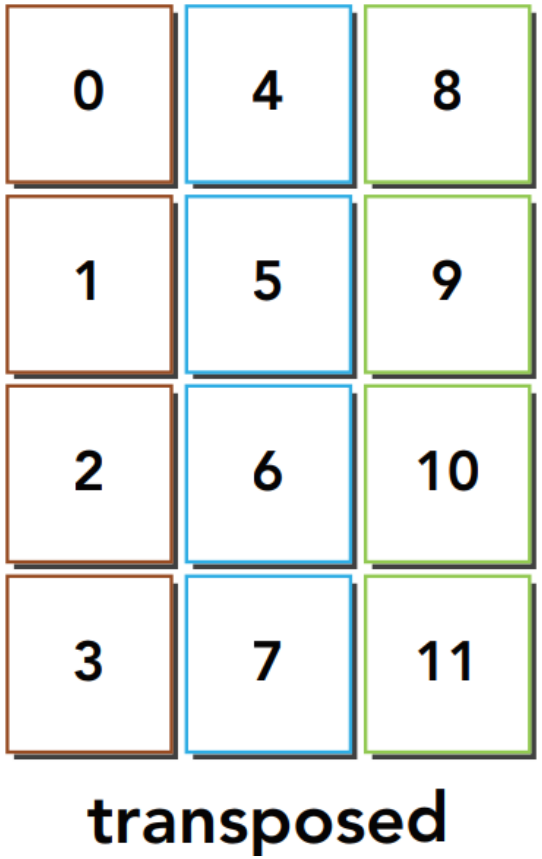
$$\text{effective bandwidth (GB/s)} = \frac{(\text{bytes read} + \text{bytes written}) \times 10^{-9}}{\text{time elapsed}}$$

4.2 Matrix Transpose Problem

Matrix Transpose Problem

- Taking the transpose of a matrix implies exchanging each row with the corresponding column.



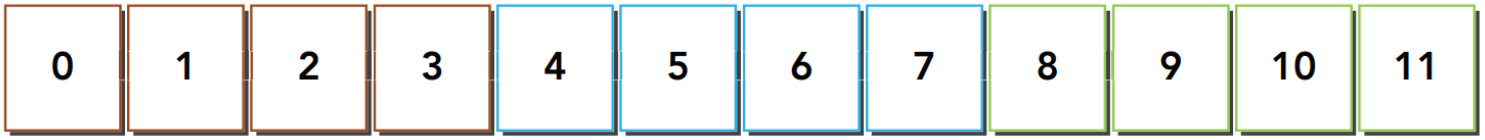


Host-based Implementation

- Suppose the matrix is stored in a 1D array.
  - nx rows * ny columns

```
void transposeHost(float *out, float *in, const int nx, const int ny) {
   for (int iy = 0; iy < ny; ++iy) {
      for (int ix = 0; ix < nx; ++ix) {
         out[ix*ny+iy] = in[iy*nx+ix];
      }
   }
}
```

data layout of original matrix



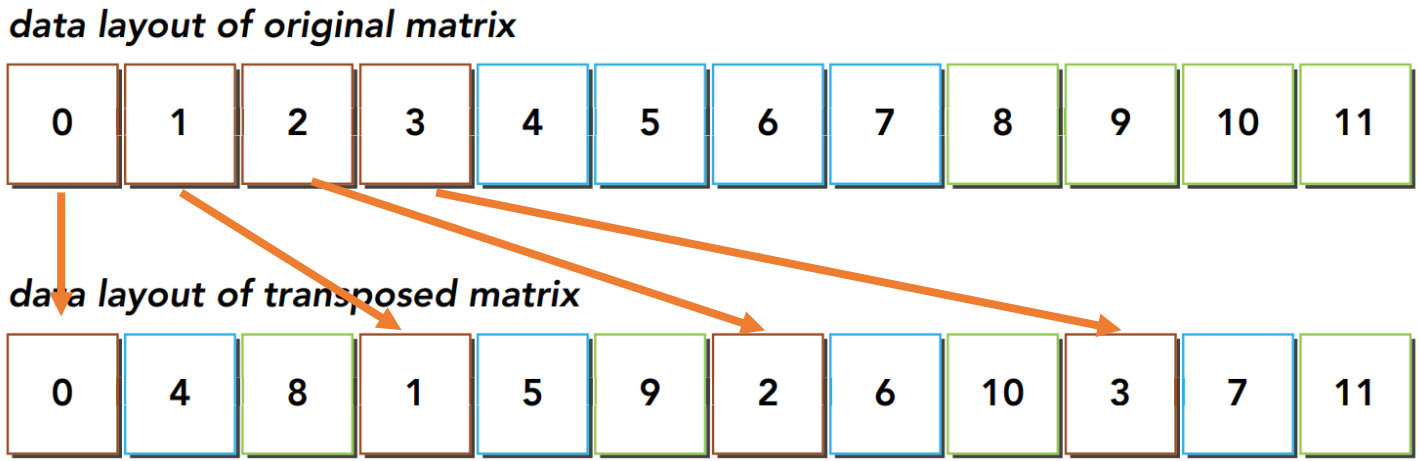
data layout of transposed matrix



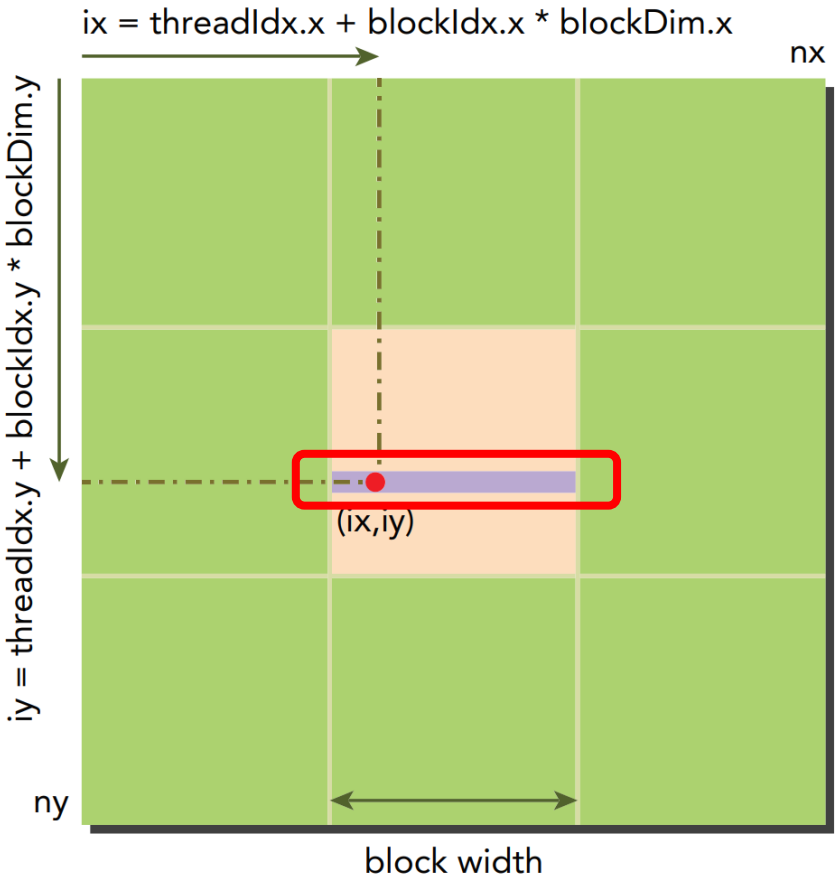
Observations

```
for (int iy = 0; iy < ny; ++iy) {
   for (int ix = 0; ix < nx; ++ix) {
      out[ix*ny+iy] = in[iy*nx+ix];
   }
}
```

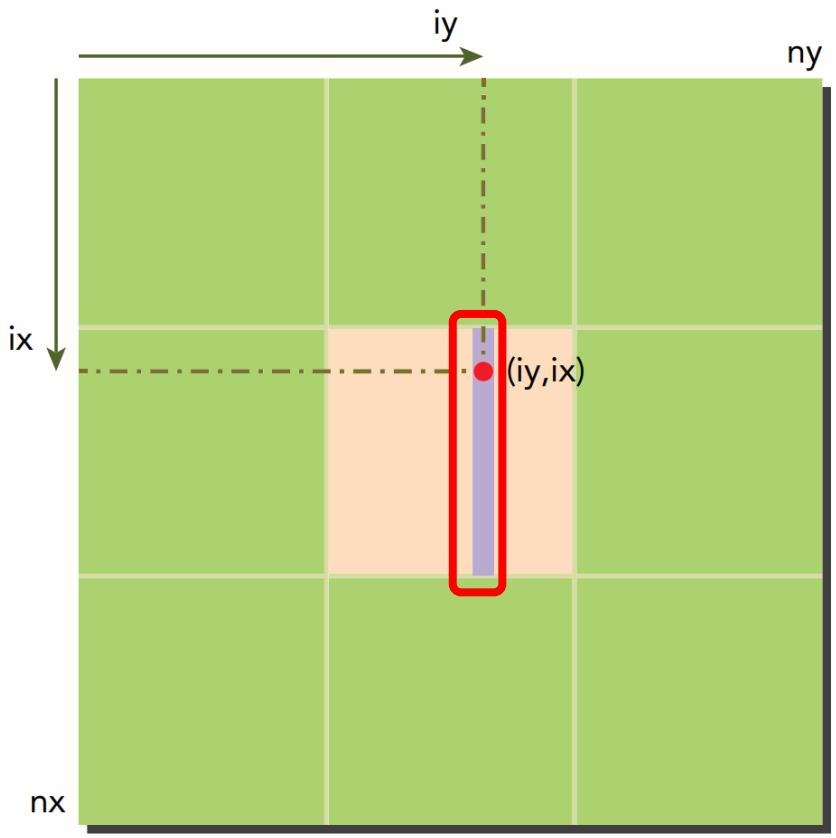
- **Reads**:
  - accessed by rows in the original matrix;
  - results in coalesced access.
- **Writes**:
  - accessed by columns in the transposed matrix;
  - results in strided access.



Two Versions of Transpose Kernel



(1) reads by rows and stores by columns



(2) reads by columns and stores by rows

Upper and Lower Performance Bound

- create two copy kernels to calculate rough upper and lower bounds for all transpose kernels

KERNEL	BANDWIDTH	RATIO TO PEAK BANDWIDTH
Theoretical peak bandwidth	177.6 GB/s	
`CopyRow` : Load/store using rows	125.67 GB/s	Upper bound: 70.76%
`CopyCol` : Load/store using columns	58.76 GB/s	Lower bound: 33.09%



transpose.cu

Naive Transpose GPU Kernels

- Reading Rows vs. Reading Columns

```
__global__ void transposeNaiveRow(float *out, float *in, const int nx,
      const int ny) {
   unsigned int ix = blockDim.x * blockIdx.x + threadIdx.x;
   unsigned int iy = blockDim.y * blockIdx.y + threadIdx.y;

   if (ix < nx && iy < ny) {
      out[ix * ny + iy] = in[iy * nx + ix];
   }
}
```

```
__global__ void transposeNaiveCol(float *out, float *in, const int nx,
      const int ny) {
   unsigned int ix = blockDim.x * blockIdx.x + threadIdx.x;
   unsigned int iy = blockDim.y * blockIdx.y + threadIdx.y;

   if (ix < nx && iy < ny) {
      out[iy*nx + ix] = in[ix*ny + iy];
   }
}
```

Naive Transpose GPU Kernels

TABLE 4-5: Effective Bandwidth of Kernels (L1 Cache Enabled)

KERNEL	BANDWIDTH	RATIO TO PEAK BANDWIDTH
`NaiveRow` : Load rows/store columns	64.16 GB/s	36.13%
`NaiveCol` : Load columns/store rows	81.64 GB/s	45.97%

NaiveCol performs better as the strided reads are cached

TABLE 4-6: Effective Bandwidth of Kernels (L1 Cache Disabled)

KERNEL	BANDWIDTH	NOTES
`CopyRow` : Load/store using rows	128.07 GB/s	Upper bound
`CopyCol` : Load/store using columns	40.42 GB/s	Lower bound
`NaiveRow` : Load rows/store columns	63.79 GB/s	Strided write/coalesced read
`NaiveCol` : Load columns/store rows	47.13 GB/s	Strided read/coalesced write

disabling load caching impacts strided read

TABLE 4-7: Load/Store Throughput of Kernels (L1 Cache Enabled)

KERNEL	LOAD THROUGHPUT	STORE THROUGHPUT
`CopyRow` : Load/store using rows	131.46 GB/s	65.32 GB/s
`CopyCol` : Load/store using columns	475.67 GB/s	118.52 GB/s
`NaiveRow` : Load rows/store columns	129.05 GB/s	64.31 GB/s
`NaiveCol` : Load columns/store rows	642.33 GB/s	40.02 GB/s

highest load throughput is obtained with cached, strided reads.

TABLE 4-8: Load/Store Efficiency of Kernels (L1 Cache Enabled)

KERNEL	LOAD EFFICIENCY	STORE EFFICIENCY
`CopyRow` : Load/store using rows	49.81%	100.00%
`CopyCol` : Load/store using columns	6.23%	25.00%
`NaiveRow` : Load rows/store columns	50.00%	25.00%
`NaiveCol` : Load columns/store rows	6.21%	100.00%

Even with poor load efficiency, caching of loads in L1 cache can limit the negative performance impact of strided loads.

5. Matrix Addition with Unified Memory

Matrix Addition with Unified Memory

- To simplify managing separate host and device memory spaces
  - Replace host and device memory allocations with managed memory allocations to eliminate duplicate pointers.
  - Remove all explicit memory copies

```
float *A, *B, *gpuRef;
cudaMallocManaged((void **)&A, nBytes);
cudaMallocManaged((void **)&B, nBytes);
cudaMallocManaged((void **)&gpuRef,  nBytes);

initialData(A, nxy);
initialData(B, nxy);

sumMatrixGPU<<<grid, block>>>(A, B, gpuRef, nx, ny);
cudaDeviceSynchronize();
```

declaring and allocating three managed arrays

initialize the two input matrices on the host

invoke the matrix addition kernel with the pointers to managed memory

Performance Comparison

- Programs With vs. Without Managed Memory

KERNEL OR RUNTIME FUNCTIONS	WITH MANAGED MEMORY	WITHOUT MANAGED MEMORY
Matrix Kernel	1.6114	1.6024
CUDA Kernel Launch	49.259	70.717
CUDA memcpy HtoD		37.987
CUDA memcpy DtoH		20.252

kernel performance with managed memory is nearly as fast as explicitly copying data between host and device, and requires significantly less programming effort.

Summary

Summary

- **Global memory** is the **largest, highest-latency**, and most commonly used memory.
- **Two guidelines** for **improving bandwidth utilization**:
  - Maximize the number of concurrent memory accesses in-flight.
  - Maximize the utilization of bytes that travel on the bus between global memory and SM on-chip memory.
- **Create more independent memory requests** in each thread using unrolling or exposing sufficient parallelism.
- Strive for **aligned and coalesced memory accesses**.
- **Unified Memory** greatly **simplifies** CUDA programming
  - eliminating duplicate pointers
  - no need to explicitly transfer data between the host and device.